

Cambiar el tono sin alterar la duración

Phase Vocoder (Dolson 1986) + Miku Stomp digital

Procesamiento Digital de Señales e Imágenes (INFB6063) - UTEM 2026-1

Francisco Alejandro Pinto Abraham - RUT 21.571.239-7

Artículo: M. Dolson, "The Phase Vocoder: A Tutorial", CMJ 10(4), 1986

github.com/k0ngan/mikustomp

1. El problema

- Subir o bajar el tono de un sonido (afinar, transponer, armonizar).
- La vía ingenua (remuestreo) sube el tono PERO:
 - cambia la duración (acorta o alarga el audio),
 - corre los formantes -> efecto "ardilla".
- Objetivo: desacoplar tono, duración y timbre con DSP clásico.

2. El artículo: Dolson (1986)

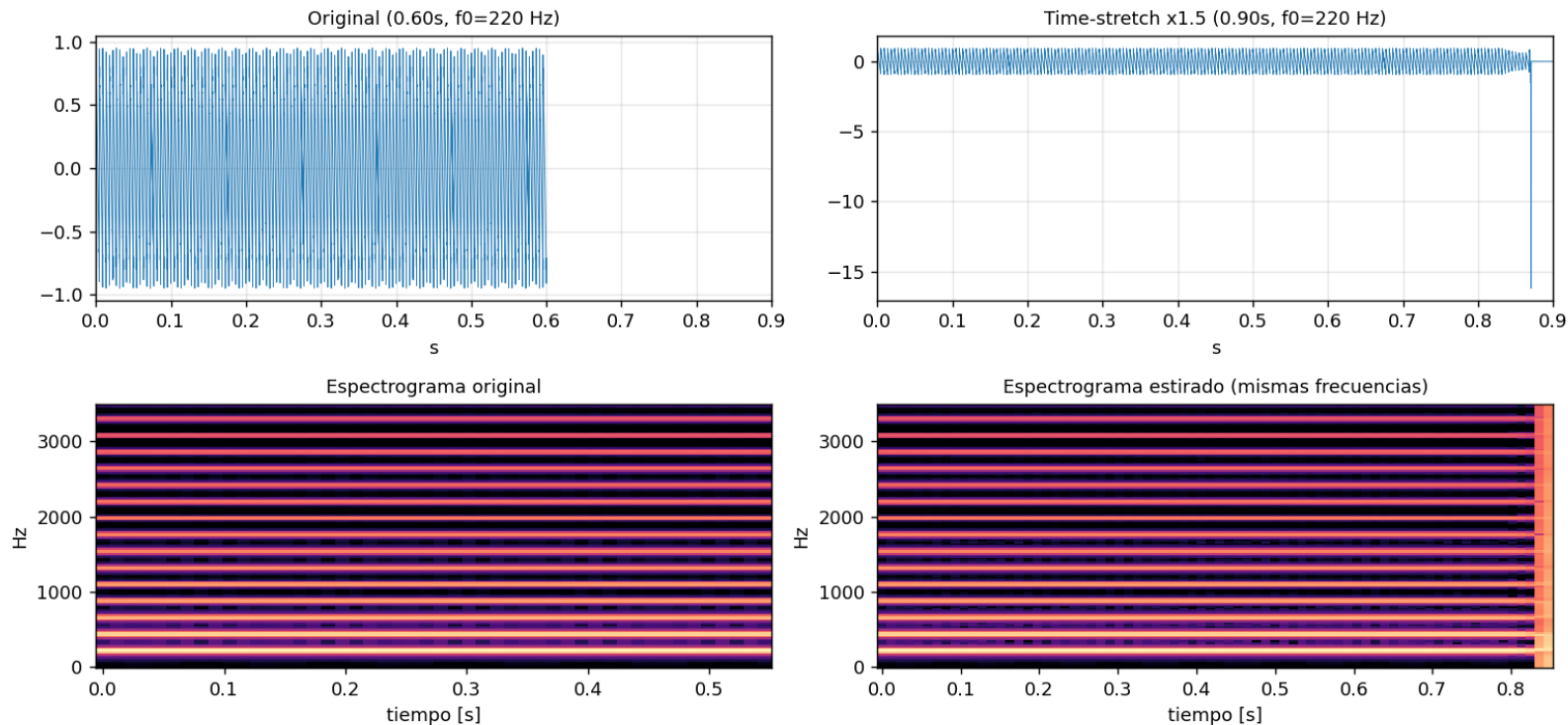
- Analiza la señal con la STFT: por banda, separa magnitud y FASE.
- La fase da la FRECUENCIA INSTANTÁNEA (diferencia de fase entre tramas, mod 2π).
- Time-stretch: re-sintetizar con un salto distinto, acumulando la fase $\rightarrow$ cambia
 - la duración sin tocar el tono.
- Pitch-shift = time-stretch + remuestreo.
- Limitaciones que reporta: phasiness y emborronado de transitorios.

3. Relación con el curso

- STFT y enventanado Hann -> Unit 2 - L1
- Espectro, magnitud y fase de la DFT -> Unit 1 - L2/L3
- Frecuencia instantánea (fase, unwrapping) -> Unit 1 - L2
- Síntesis por superposición (overlap-add) -> Unit 2 - L1/L2
- Pitch-shift = stretch + remuestreo -> Unit 1 - L3
- Preservación de formantes (máscara $H[k]$) + cepstrum -> Unit 2 - L3
- Todo DSP clásico: sin deep learning / IA generativa.

4. Reproducción: duración cambia, tono no

Reproducción phase vocoder (Dolson 1986): la duración cambia, el tono NO

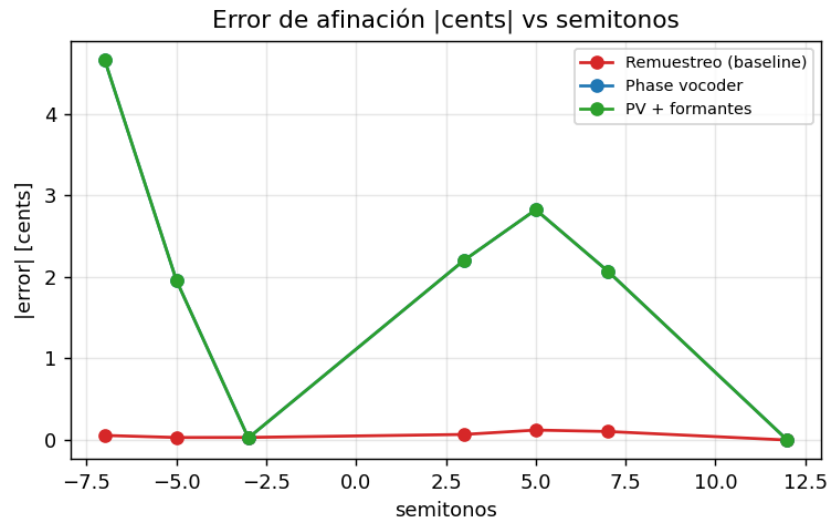
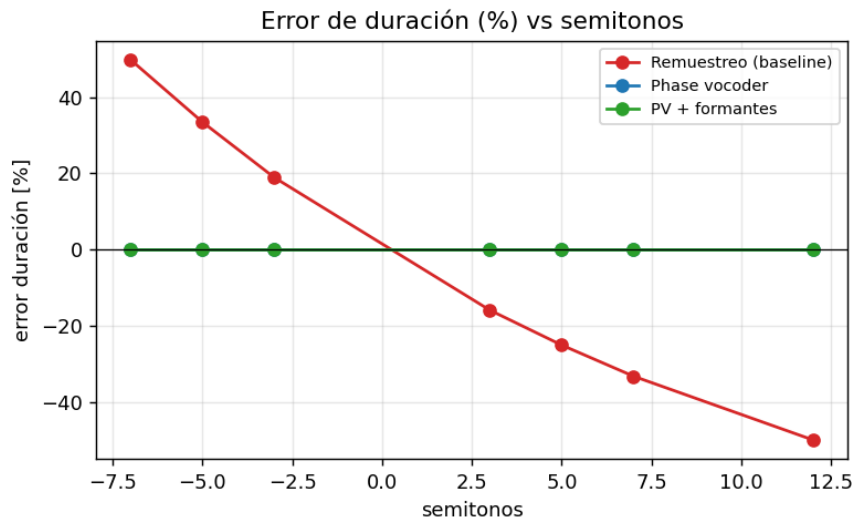


5. Extensión propia: PV + formantes

- El pitch-shift básico mueve los formantes junto con los armónicos.
- Idea: re-imponer la ENVOLVENTE espectral original tras subir el tono.
- Se estima la envolvente por CEPSTRUM (lifter de baja frecuencia).
- Máscara de corrección: $H[k] = \text{env_original} / \text{env_desplazada}$ (limitada a ± 60 dB).
- Resultado: los armónicos suben, pero los formantes (timbre) quedan fijos.
- Es filtrado en frecuencia (Unit 2 - L3) guiado por cepstrum. Sin ML.

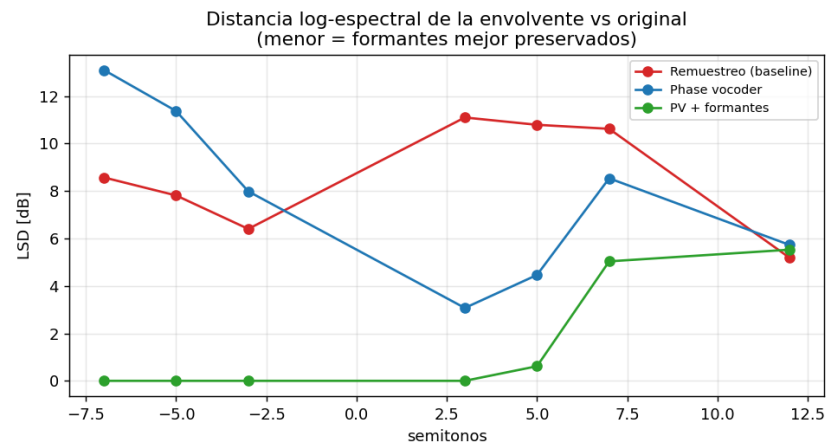
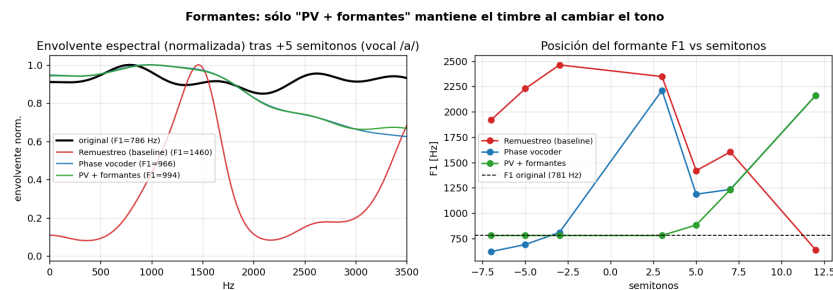
6. Resultados: tono y duración

Tono y duración: el remuestreo cambia el largo; el phase vocoder lo conserva



Remuestreo: hasta +-50% de error de duración | Phase vocoder: 0% (afinación < 5 cents)

7. Resultados: formantes

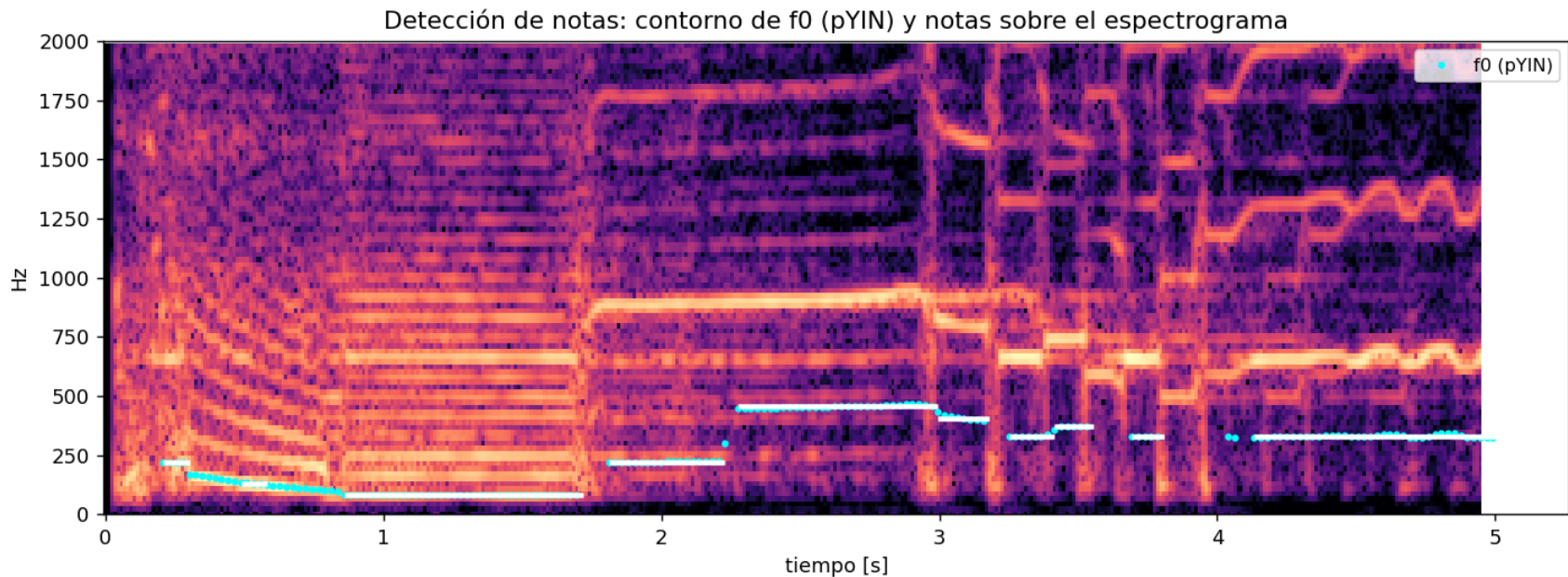


Sólo PV + formantes mantiene F1 y baja la distorsión de la envolvente: ~8.6 dB -> ~1.6 dB

8. Aplicación: Miku Stomp digital

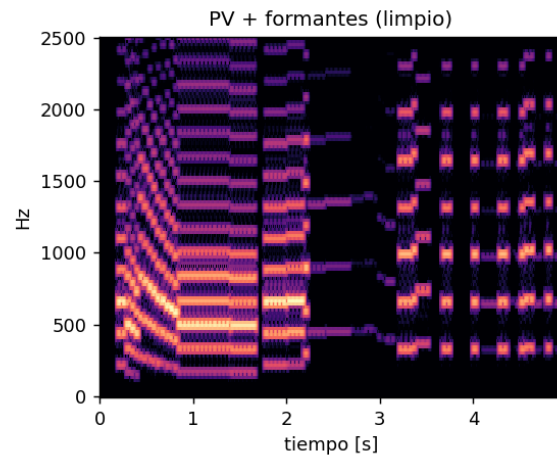
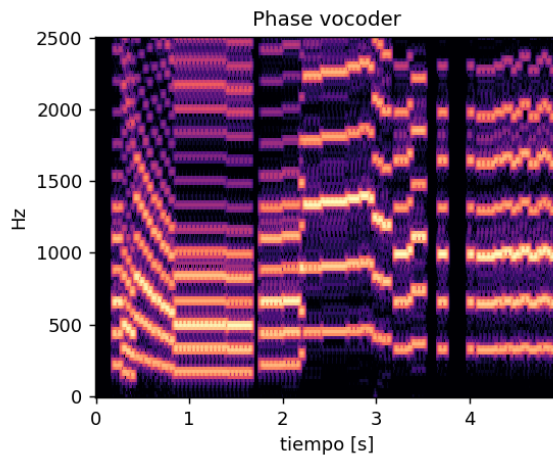
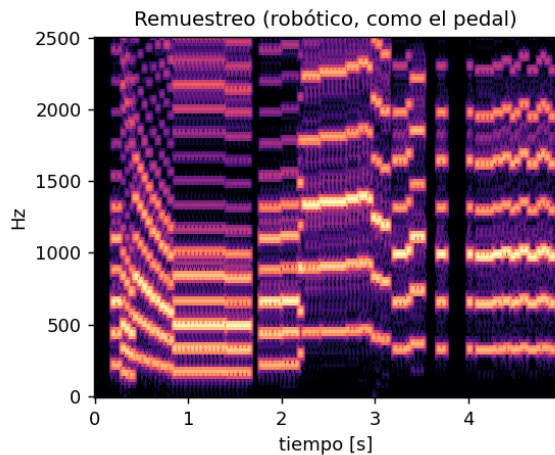
- Replica del pedal Korg Miku Stomp en Python (DSP puro).
- Flujo: guitarra -> detectar la nota (pYIN) -> afinar una voz a esa nota -> mezclar.
- Usa nuestro pitch-shift CON formantes: la voz mantiene su timbre al seguir la melodía.
- Mejor que el pedal: el Korg corre los formantes (ardilla); aquí no.
- Voz real incluida (Vocaloid 4 CyberDiva); pYIN es DSP clásico, no una red neuronal.
- Cuaderno autocontenido para Google Colab (subir audio -> procesar -> descargar).

9. Miku Stomp: detección de notas (pYIN)



10. Miku Stomp: 3 modos (robótico vs limpio)

Miku Stomp: voz sintetizada por método (misma melodía de guitarra)



11. Discusión y conclusiones

- Funciona: desplazamientos moderados (-7 a +3 semitonos) en sonidos cuasi-estacionarios.
- Falla: saltos grandes ($\geq +5/+7$) degradan la corrección de formantes; transitorios borrosos.
- Conceptos clave: STFT, fase/frecuencia instantánea, overlap-add, cepstrum, muestreo.
- La extensión MEJORÓ el timbre (LSD $\sim 8.6 \rightarrow \sim 1.6$ dB) conservando la duración.
- Trabajo futuro: phase-locking (Laroche-Dolson), TD-PSOLA, polifonía/RVC (solo mención).
- Código, informe, audios y cuaderno: github.com/k0ngan/mikustromp